



Министерство науки и высшего образования Российской
Федерации
Федеральное государственное автономное
образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет: «Информатика и системы управления» (ИУ)

Кафедра: «Информационная безопасность» (ИУ8)

Домашнее задание № 3

по курсу «Математические основы искусственного
интеллекта»

Выполнил: Бородин Г.В.

Группа: ИУ8-13М

Проверил: Каганов Ю.Т.

Цель работы:

1. Изучение алгоритмов многомерного поиска 1-го и 2-го порядка.
2. Разработка программ реализации алгоритмов многомерного поиска 1-го и 2-го порядка.
3. Вычисление экстремумов функции.

1)

- a)
- b)
- c)
- d)

2)

Тестовая функция Розенброка для варианта 1:

$$f(x) = 50 \cdot (x_1^2 - x_2)^2 + 2(x_1 - 1)^2 + 10$$

Реализация функции Розенброка:

```
# -----  
# функция Розенброка  
# -----  
def rosenbrock(x):  
    x1, x2 = x  
    return 50 * (x1**2 - x2)**2 + 2 * (x1 - 1)**2 + 10  
  
def grad_rosenbrock(x):  
    x1, x2 = x  
    df_dx1 = 200 * x1 * (x1**2 - x2) + 4 * (x1 - 1)  
    df_dx2 = -100 * (x1**2 - x2)  
    return np.array([df_dx1, df_dx2])  
  
def hess_rosenbrock(x):  
    x1, x2 = x  
    d2f_dx1x1 = 200 * (x1**2 - x2) + 400 * x1**2 + 4  
    d2f_dx1x2 = -200 * x1  
    d2f_dx2x2 = 100  
    return np.array([[d2f_dx1x1, d2f_dx1x2],  
                     [d2f_dx1x2, d2f_dx2x2]])
```

Рисунок 1 - Функция Розенброка в python

Метод наискорейшего спуска

```
#-----
def generic_optimize(method_func, name):
    x0 = [-1.2, 1.0]
    x_opt, traj = method_func(rosenbrock, grad_rosenbrock, x0)
    f_val = rosenbrock(x_opt)
    print(f"{name:20}: x* = [{x_opt[0]:.6f}, {x_opt[1]:.6f}], f = {f_val:.2e}, итераций = {len(traj)-1}")
    return np.array(traj)

def steepest_descent(f, grad_f, x0, tol=1e-6, max_iter=500):
    x = np.array(x0, dtype=float)
    trajectory = [x.copy()]
    for k in range(max_iter):
        g = grad_f(x)
        if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
            break
        d = -g
        alpha = line_search_armijo(f, x, g, d)
        x = x + alpha * d
        trajectory.append(x.copy())
    return x, trajectory
```

Рисунок 2 - Реализация в Python

Метод Флетчера-Ривза

```
def fletcher_reeves(f, grad_f, x0, tol=1e-6, max_iter=500):
    x = np.array(x0, dtype=float)
    g = grad_f(x)
    d = -g
    trajectory = [x.copy()]
    for k in range(max_iter):
        if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
            break
        alpha = line_search_armijo(f, x, g, d)
        x_new = x + alpha * d
        g_new = grad_f(x_new)
        beta = np.dot(g_new, g_new) / (np.dot(g, g) + 1e-12)
        d = -g_new + beta * d
        x, g = x_new, g_new
        trajectory.append(x.copy())
    return x, trajectory
```

Рисунок 3 - Реализация в Python

Метод Полака-Ревьера:

```
def polak_ribiere(f, grad_f, x0, tol=1e-6, max_iter=500):
    x = np.array(x0, dtype=float)
    g = grad_f(x)
    d = -g
    trajectory = [x.copy()]
    for k in range(max_iter):
        if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
            break
        alpha = line_search_armijo(f, x, g, d)
        x_new = x + alpha * d
        g_new = grad_f(x_new)
        beta = np.dot(g_new, g_new - g) / (np.dot(g, g) + 1e-12)
        beta = max(beta, 0)
        d = -g_new + beta * d
        x, g = x_new, g_new
        trajectory.append(x.copy())
    return x, trajectory
```

Рисунок 4 - Реализация в Python

Метод Девидона-Флетчера-Пауэлла

```

def dfp(f, grad_f, x0, tol=1e-6, max_iter=500):
    n = len(x0)
    x = np.array(x0, dtype=float)
    g = grad_f(x)
    H = np.eye(n)
    trajectory = [x.copy()]
    for k in range(max_iter):
        if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
            break
        d = -H @ g
        alpha = line_search_armijo(f, x, g, d)
        x_new = x + alpha * d
        g_new = grad_f(x_new)
        s = x_new - x
        y = g_new - g
        sy = np.dot(s, y)
        if abs(sy) < 1e-12:
            break
        Hy = H @ y
        H = H + np.outer(s, s) / sy - np.outer(Hy, Hy) / (np.dot(y, Hy) + 1e-12)
        x, g = x_new, g_new
        trajectory.append(x.copy())
    return x, trajectory

```

Рисунок 5 - Реализация в Python

Метод Левенберга-Марквардта

```

def levenberg_marquardt(f, grad_f, hess_f, x0, tol=1e-6, max_iter=500):
    x = np.array(x0, dtype=float)
    trajectory = [x.copy()]
    nu = 2.0
    for k in range(max_iter):
        g = grad_f(x)
        if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
            break
        H = hess_f(x)
        lambda_ = 1e-3
        while True:
            try:
                A = H + lambda_ * np.eye(len(x))
                d = np.linalg.solve(A, -g)
                x_new = x + d
                if f(x_new) < f(x):
                    x = x_new
                    lambda_ = max(lambda_ / nu, 1e-12)
                    break
            except:
                lambda_ *= nu
        trajectory.append(x.copy())
    return x, trajectory

```

Рисунок 6 - Реализация в Python

Результаты:

Код программы реализует сравнение методов путем анимаций на 2D и 3D графиков при помощи траекторий. На рисунке 7 видно, что метод градиентного спуска выполняется очень долго, что он не закончил работу за 10 секунд.

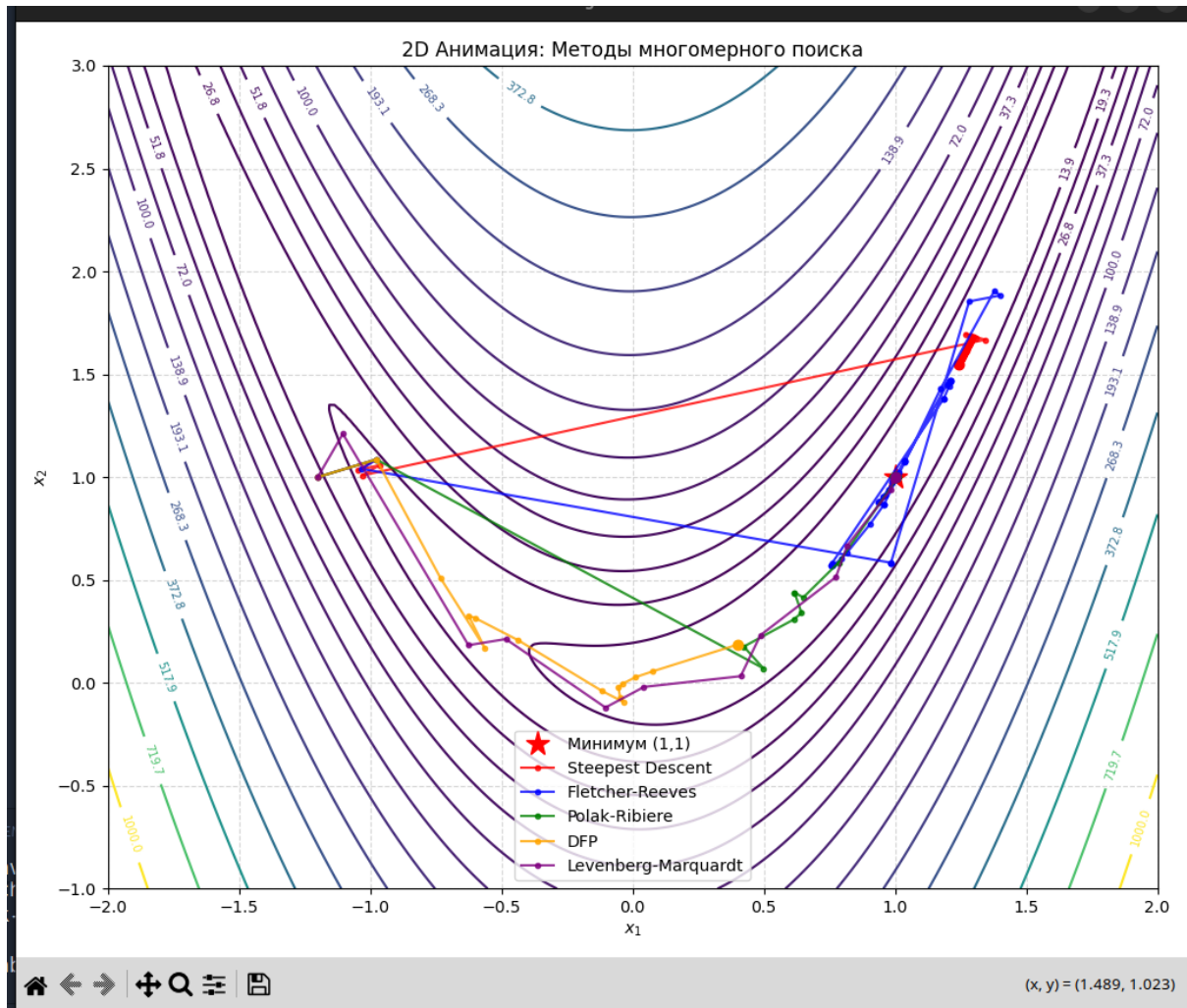


Рисунок 7 – 2D анимация

3D Анимация: Методы многомерного поиска

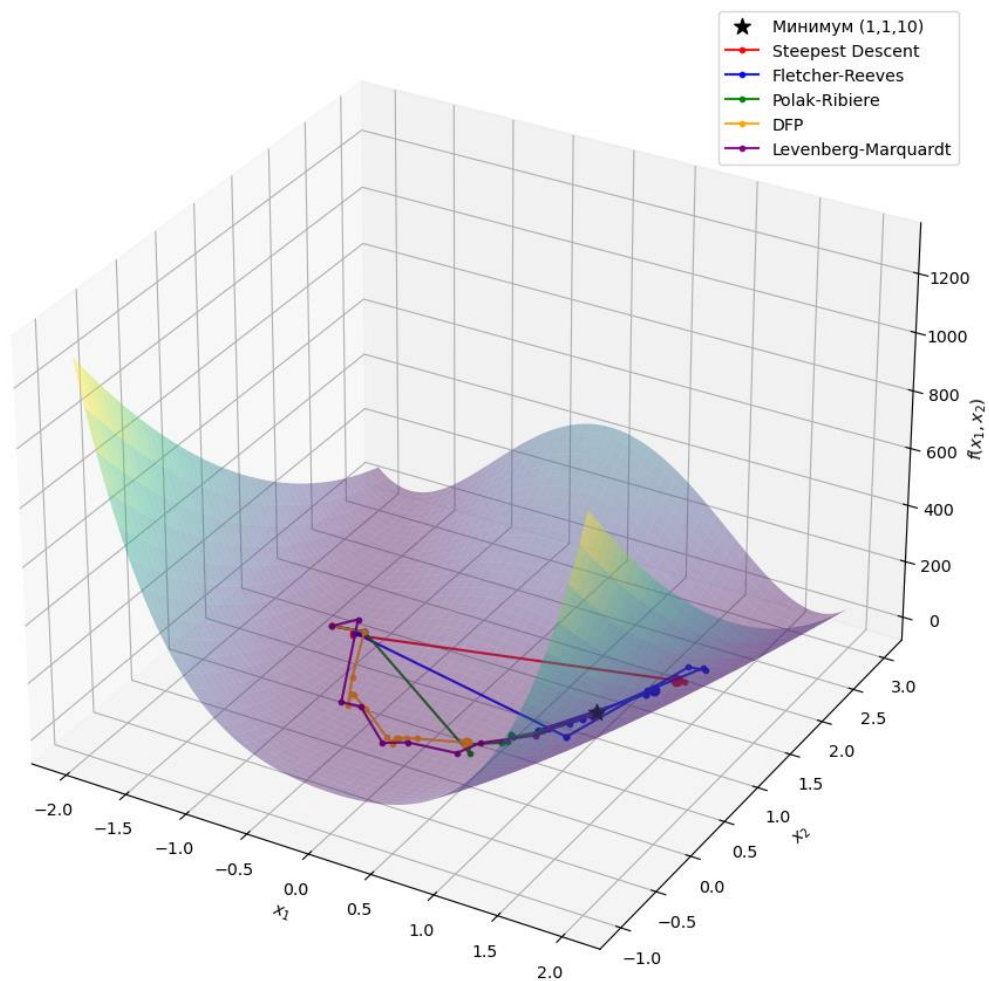


Рисунок 8 – 3D анимация

```
Steepest Descent : x* = [1.071263, 1.148872], f = 1.00e+01, итераций = 500
Fletcher-Reeves : x* = [1.000000, 1.000000], f = 1.00e+01, итераций = 273
Polak-Ribiere : x* = [1.000381, 1.000840], f = 1.00e+01, итераций = 500
DFP : x* = [0.398969, 0.185715], f = 1.08e+01, итераций = 14
Levenberg-Marquardt : x* = [1.000000, 1.000000], f = 1.00e+01, итераций = 13
```

Рисунок 9 - Сходимость и результаты

Приложение А:

Код программы

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter
from mpl_toolkits.mplot3d import Axes3D

# -----
# функция Розенброка
# -----

def rosenbrock(x):
    x1, x2 = x
```

```

return 50 * (x1**2 - x2)**2 + 2 * (x1 - 1)**2 + 10

def grad_rosenbrock(x):
    x1, x2 = x
    df_dx1 = 200 * x1 * (x1**2 - x2) + 4 * (x1 - 1)
    df_dx2 = -100 * (x1**2 - x2)
    return np.array([df_dx1, df_dx2])

def hess_rosenbrock(x):
    x1, x2 = x
    d2f_dx1x1 = 200 * (x1**2 - x2) + 400 * x1**2 + 4
    d2f_dx1x2 = -200 * x1
    d2f_dx2x2 = 100
    return np.array([[d2f_dx1x1, d2f_dx1x2],
                    [d2f_dx1x2, d2f_dx2x2]])

# -----
# Backtracking line search (Армико)
# -----
def line_search_armijo(f, x, grad, direction, alpha0=1.0, rho=0.5, c=1e-4, max_iter=30):
    alpha = alpha0
    f_x = f(x)
    for _ in range(max_iter):
        x_new = x + alpha * direction
        if np.any(np.isnan(x_new)) or np.any(np.isinf(x_new)):
            alpha *= rho
            continue
        if f(x_new) <= f_x + c * alpha * np.dot(grad, direction):
            return alpha
        alpha *= rho
    return alpha

# -----
# Методы оптимизации (с защитой)
# -----
def generic_optimize(method_func, name):
    x0 = [-1.2, 1.0]
    x_opt, traj = method_func(rosenbrock, grad_rosenbrock, x0)
    f_val = rosenbrock(x_opt)
    print(f"{name:20}: x* = [{x_opt[0]:.6f}, {x_opt[1]:.6f}], f = {f_val:.2e}, итераций = {len(traj)-1}")
    return np.array(traj)

def steepest_descent(f, grad_f, x0, tol=1e-6, max_iter=500):
    x = np.array(x0, dtype=float)
    trajectory = [x.copy()]
    for k in range(max_iter):

```

```

g = grad_f(x)
if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
    break
d = -g
alpha = line_search_armijo(f, x, g, d)
x = x + alpha * d
trajectory.append(x.copy())
return x, trajectory

def fletcher_reeves(f, grad_f, x0, tol=1e-6, max_iter=500):
x = np.array(x0, dtype=float)
g = grad_f(x)
d = -g
trajectory = [x.copy()]
for k in range(max_iter):
if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
    break
alpha = line_search_armijo(f, x, g, d)
x_new = x + alpha * d
g_new = grad_f(x_new)
beta = np.dot(g_new, g_new) / (np.dot(g, g) + 1e-12)
d = -g_new + beta * d
x, g = x_new, g_new
trajectory.append(x.copy())
return x, trajectory

def polak_ribiere(f, grad_f, x0, tol=1e-6, max_iter=500):
x = np.array(x0, dtype=float)
g = grad_f(x)
d = -g
trajectory = [x.copy()]
for k in range(max_iter):
if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
    break
alpha = line_search_armijo(f, x, g, d)
x_new = x + alpha * d
g_new = grad_f(x_new)
beta = np.dot(g_new, g_new - g) / (np.dot(g, g) + 1e-12)
beta = max(beta, 0)
d = -g_new + beta * d
x, g = x_new, g_new
trajectory.append(x.copy())
return x, trajectory

def dfp(f, grad_f, x0, tol=1e-6, max_iter=500):
n = len(x0)
x = np.array(x0, dtype=float)

```

```

g = grad_f(x)
H = np.eye(n)
trajectory = [x.copy()]
for k in range(max_iter):
    if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
        break
    d = -H @ g
    alpha = line_search_armijo(f, x, g, d)
    x_new = x + alpha * d
    g_new = grad_f(x_new)
    s = x_new - x
    y = g_new - g
    sy = np.dot(s, y)
    if abs(sy) < 1e-12:
        break
    Hy = H @ y
    H = H + np.outer(s, s) / sy - np.outer(Hy, Hy) / (np.dot(y, Hy) + 1e-12)
    x, g = x_new, g_new
    trajectory.append(x.copy())
return x, trajectory

def levenberg_marquardt(f, grad_f, hess_f, x0, tol=1e-6, max_iter=500):
    x = np.array(x0, dtype=float)
    trajectory = [x.copy()]
    nu = 2.0
    for k in range(max_iter):
        g = grad_f(x)
        if np.linalg.norm(g) < tol or np.any(np.isnan(g)) or np.any(np.isinf(g)):
            break
        H = hess_f(x)
        lambda_ = 1e-3
        while True:
            try:
                A = H + lambda_ * np.eye(len(x))
                d = np.linalg.solve(A, -g)
                x_new = x + d
                if f(x_new) < f(x):
                    x = x_new
                    lambda_ = max(lambda_ / nu, 1e-12)
                    break
            except:
                lambda_ *= nu
        except np.linalg.LinAlgError:
            lambda_ *= nu
        trajectory.append(x.copy())
    return x, trajectory

```

```

# -----
# Запуск всех методов
# -----
trajectories = {}
methods = {
    "Steepest Descent": steepest_descent,
    "Fletcher-Reeves": fletcher_reeves,
    "Polak-Ribiere": polak_ribiere,
    "DFP": dfp,
    "Levenberg-Marquardt": lambda f, g, x0, **kw: levenberg_marquardt(f, g, hess_rosenbrock, x0, **kw)
}

print("Оптимизация...")
for name, method in methods.items():
    if name == "Levenberg-Marquardt":
        traj = generic_optimize(lambda f, g, x0, **kw: method(f, g, x0, **kw), name)
    else:
        traj = generic_optimize(method, name)
    trajectories[name] = traj

# -----
# Подготовка данных для графиков
# -----
x1 = np.linspace(-2, 2, 200)
x2 = np.linspace(-1, 3, 200)
X1, X2 = np.meshgrid(x1, x2)
Z = np.array([rosenbrock([x1i, x2i]) for x1i, x2i in zip(X1.ravel(), X2.ravel())]).reshape(X1.shape)

colors = ['red', 'blue', 'green', 'orange', 'purple']
names = list(trajectories.keys())

# =====
# 2D АНИМАЦИЯ
# =====
fig2d, ax2d = plt.subplots(figsize=(10, 8))
contour = ax2d.contour(X1, X2, Z, levels=np.logspace(1, 3, 15), cmap='viridis')
ax2d.clabel(contour, inline=True, fontsize=7)
ax2d.plot(1, 1, 'r*', markersize=15, label="Минимум (1,1)")

lines2d = {}
points2d = {}
for name, color in zip(names, colors):
    line, = ax2d.plot([], [], 'o-', color=color, label=name, markersize=3, alpha=0.8)
    point, = ax2d.plot([], [], 'o', color=color, markersize=6)
    lines2d[name] = line
    points2d[name] = point

```

```

ax2d.set_xlabel("$x_1$")
ax2d.set_ylabel("$x_2$")
ax2d.set_title("2D Анимация: Методы многомерного поиска")
ax2d.legend()
ax2d.grid(True, linestyle='--', alpha=0.5)

max_frames = max(len(traj) for traj in trajectories.values())

def animate2d(frame):
    for name in names:
        traj = trajectories[name]
        if frame < len(traj):
            # Траектория до текущего кадра
            x_data = traj[:frame+1, 0]
            y_data = traj[:frame+1, 1]
            lines2d[name].set_data(x_data, y_data)
            # Текущая точка — ОБЯЗАТЕЛЬНО как последовательность
            points2d[name].set_data([traj[frame, 0]], [traj[frame, 1]])
        else:
            x_data = traj[:, 0]
            y_data = traj[:, 1]
            lines2d[name].set_data(x_data, y_data)
            points2d[name].set_data([traj[-1, 0]], [traj[-1, 1]])

anim2d = FuncAnimation(fig2d, animate2d, frames=max_frames, interval=300, repeat=False)
plt.tight_layout()
plt.show()

# Сохранить 2D анимацию (раскомментируйте, если нужно)
# anim2d.save("optimization_2d.gif", writer=PillowWriter(fps=2))

# =====
# 3D АНИМАЦИЯ
# =====

fig3d = plt.figure(figsize=(12, 9))
ax3d = fig3d.add_subplot(111, projection='3d')

# Поверхность (с пониженной прозрачностью для читаемости)
ax3d.plot_surface(X1, X2, Z, cmap='viridis', alpha=0.4, linewidth=0, antialiased=True)
ax3d.scatter([1], [1], [10], color='black', s=100, marker='*', label="Минимум (1,1,10)")

lines3d = {}
points3d = {}
for name, color in zip(names, colors):
    line, = ax3d.plot([], [], [], 'o-', color=color, label=name, markersize=3, alpha=0.9)

```

```

point, = ax3d.plot([], [], [], 'o', color=color, markersize=6)
lines3d[name] = line
points3d[name] = point

ax3d.set_xlabel("$x_1$")
ax3d.set_ylabel("$x_2$")
ax3d.set_zlabel("$f(x_1, x_2)$")
ax3d.set_title("3D Анимация: Методы многомерного поиска")
ax3d.legend()

def animate3d(frame):
    for name in names:
        traj = trajectories[name]
        if frame < len(traj):
            x_traj = traj[:frame+1, 0]
            y_traj = traj[:frame+1, 1]
            z_traj = [rosenbrock(p) for p in traj[:frame+1]]
            lines3d[name].set_data(x_traj, y_traj)
            lines3d[name].set_3d_properties(z_traj)
            points3d[name].set_data([traj[frame, 0]], [traj[frame, 1]])
            points3d[name].set_3d_properties([rosenbrock(traj[frame])])
        else:
            x_traj = traj[:, 0]
            y_traj = traj[:, 1]
            z_traj = [rosenbrock(p) for p in traj]
            lines3d[name].set_data(x_traj, y_traj)
            lines3d[name].set_3d_properties(z_traj)
            points3d[name].set_data([traj[-1, 0]], [traj[-1, 1]])
            points3d[name].set_3d_properties([rosenbrock(traj[-1])])

anim3d = FuncAnimation(fig3d, animate3d, frames=max_frames, interval=400, repeat=False)
plt.tight_layout()
plt.show()

```